Carrot

System Architecture & Flow Diagrams

Personal finance management for Nigeria — Next.js 16 · Supabase · Mono Open Banking

CONTENTS

01 **Application Architecture** — Client, server, and external-service topology

02 **Database Architecture** — Full Postgres schema, relationships & RLS

03 **Data Flow** — The account-link & transaction-sync pipeline

04 **Process Flow** — Auto-categorization & the rule-learning feedback loop

05 **User Flow** — Onboarding through to every core surface of the app

Repository: github.com/aduferobiu/carrot · Deployed: carrot-ebon.vercel.app

Generated August 2026 · reflects the schema and route handlers in supabase/migrations & apps/web/src

Carrot — Application Architecture

Next.js 16 App Router on Vercel · Supabase (Postgres + Auth) · Mono Open Banking API

Mono Connect widget (connect.js) — hosted bank-login UI → returns a one-time code

CLIENT — BROWSER

App Router Pages — /(app)/*

Dashboard, Transactions, Budgets, Insights,
Subscriptions, Accounts, Settings — all "use client"

Kobo Store — React Context (store.tsx)

Single global client state: session, accounts, tx,
budgets, subscriptions, healthScore, notifications

Supabase JS Client (lib/supabase.ts)

Anon key; session persisted to localStorage;
getSession() restores it with no network call

Mono Connect (lib/mono.ts)

Loads @mono.co/connect.js on demand; opens the
hosted bank-login widget, returns a one-time code

fetch() + Authorization: Bearer <session JWT>

SERVER — NEXT.JS ROUTE HANDLERS (VERCEL)

Route Handlers — app/api/*

accounts/link, accounts/[id]/sync, transactions/[id]/category, subscriptions*, insights/health-score

auth.ts — requireAuth()

Validates the Bearer token via supabase.auth.getUser(token); 401s on failure

accountSync.ts — sync orchestrator

Fetches 6mo of Mono transactions, categorizes, upserts, runs subscription detection

categorization/* · subscriptions/detect.ts · healthScore/*

Pure in-memory business logic — see Data Flow & Process Flow diagrams

supabase.ts — service-role client

SUPABASE_SERVICE_ROLE_KEY; bypasses RLS — never imported client-side

mono-sec-key header (server-only secret)

EXTERNAL - MONO API

Mono Open Banking API

REST · api.withmono.com/v2

POST /accounts/auth
code → mono_account_id

GET /accounts/:id
name, institution, balance,
account number (masked)

GET /accounts/:id/transactions
±6 months, narration/
amount/type/date, unpaginated

Auth: mono-sec-key header
(server-only, never sent to
the browser)

supabase-js client (anon key)
RLS-scoped reads & writes

service-role client
bypasses RLS entirely

DATA — SUPABASE (POSTGRES + AUTH)

Postgres — 10 tables, Row-Level Security throughout

profiles · accounts · categories · transactions · budgets · notifications
categorization_rules · transaction_corrections · uncategorized_log · subscriptions

Every policy checks auth.uid() = user_id — the browser can only ever see its own rows.
See the Database Architecture diagram for the full column-level schema.

uncategorized_log has no client policy — written only by the service-role client.

Supabase Auth — auth.users

Issues the session JWT the browser stores and every route handler validates via requireAuth().

A trigger (on_auth_user_created) inserts a matching public.profiles row the moment a user signs up.

Reached directly by the browser (login/register, session restore) and by the server (token validation only).

Client (browser) · Server (Vercel) · Data tier (Supabase) · External service / component card · Internal request · Third-party widget (client-side)

Carrot — Database Architecture

Supabase / PostgreSQL — supabase/migrations/0001–0005 · 10 tables, all RLS-protected

managed by Supabase Auth

auth.users

id	**uuid**

profiles

id	uuid
full_name	text
currency	text
created_at	timestamptz

accounts

id	uuid
user_id	uuid → auth.users
name	text
type	text
institution_name	text
masked_number	text
balance	numeric(14,2)
currency	text
mono_account_id	text · unique
created_at	timestamptz

categories

id	uuid
user_id	uuid, null → auth.users
name	text
kind	enum income/expense
icon	text
color	text
is_default	boolean
parent_id	uuid, null → categories
created_at	timestamptz

categorization_rules

id	uuid
category_id	uuid → categories
keyword	text · unique(user,kw)
priority	int, default 100
source	seed | user_derived
user_id	uuid, null → auth.users
created_at	timestamptz

budgets

id	uuid
user_id	uuid → auth.users
category_id	uuid → categories
amount	numeric(14,2)
period_start	date
created_at	timestamptz

unique (user_id, category_id, period_start)

transactions

id	uuid
user_id	uuid → auth.users
account_id	uuid → accounts
transfer_account_id	uuid, null → accounts
category_id	uuid, null → categories
type	income/expense/transfer
amount	numeric(14,2)
description	text
raw_description	text — from Mono
normalized_description	text
category_source	rule-matched/user-corrected/fallback
matched_rule_id	uuid, null → categorization_rules
mono_transaction_id	text · unique
occurred_at	timestamptz
created_at	timestamptz

subscriptions

id	uuid
user_id	uuid → auth.users
account_id	uuid → accounts
merchant_label	text
category_id	uuid, null → categories
frequency	weekly/monthly/yearly
average_amount	numeric(14,2)
last_amount	numeric(14,2)
first_seen_at	date
last_seen_at	date
predicted_next_charge_at	date
status	active/dismissed/manual/needs_review
last_alerted_at	date
created_at	timestamptz
updated_at	timestamptz

unique (user_id, account_id, merchant_label)

uncategorized_log

id	uuid
transaction_id	uuid, null → transactions
normalized_description	text
created_at	timestamptz

server-only writes — no client RLS policy

transaction_corrections

id	uuid
user_id	uuid → auth.users
transaction_id	uuid → transactions
normalized_description	text
corrected_category_id	uuid → categories
created_at	timestamptz

notifications

id	uuid
user_id	uuid → auth.users
type	text
title	text
message	text
color	text
icon	text
is_read	boolean
created_at	timestamptz

How isolation works

Every policy is `using (auth.uid() = user_id)` — Postgres, not app code, is the only thing stopping one user from reading another's rows.

categories with user_id = null are shared defaults, visible to everyone.

Seed categorization_rules (user_id null) work the same way.

LEGEND

PK primary key — **FK** foreign key (required) — **FK** foreign key (nullable) — ——— required relationship — - - - - nullable relationship

All tables carry `enable row level security`; every policy below is scoped to auth.uid() = user_id unless noted.

Carrot — Data Flow

The account-link & transaction-sync pipeline — the single most complex flow in the app

CLIENT — Kobo Store

1. **User taps "Link account" → openMonoConnect()**
   Loads @mono.co/connect.js, opens the hosted bank-login widget
2. **User logs into their bank inside the widget**
   Entirely Mono-hosted — Carrot never sees bank credentials
3. **onSuccess(code) → POST /api/accounts/link { code }**
   requireAuth() validates the caller's session first
4. **exchangeMonoCode(code) → POST /accounts/auth**
   One-time code exchanged for a durable mono_account_id
5. **getMonoAccount(id) → name, balance, institution**
   Owner-name check: every token of the Carrot profile name
6. **must appear in the bank account name, or the link is rejected**
   Blocks linking someone else's account, order-independent match
7. **insert into accounts**
   Balance stored in naira (Mono returns kobo ÷ 100)
8. **syncTransactions(): getMonoTransactions() — last 6 months**
   Best-effort — a fetch failure here doesn't undo the link above
9. **categorizeBatch(): normalize() + match against rules**
   See the Process Flow diagram for the full matching logic
10. **upsert into transactions, deduped on mono_transaction_id**
    Fallback ("Others") rows also logged for keyword-gap review
11. **runDetection(): group by (account, description) → subscriptions**
    Stale ones flip to needs_review; renewals due soon raise an alert
12. **{ account, transactionsImported } → client refreshes store**
    Re-fetches accounts, transactions, subscriptions, notifications

MONO OPEN BANKING API

external entity — hosted widget + REST

auth.uid never leaves the server

D1 accounts

read — user rules first, then seed rules

D2 categorization_rules (read)

D3 transactions

D4 uncategorized_log

D5 subscriptions

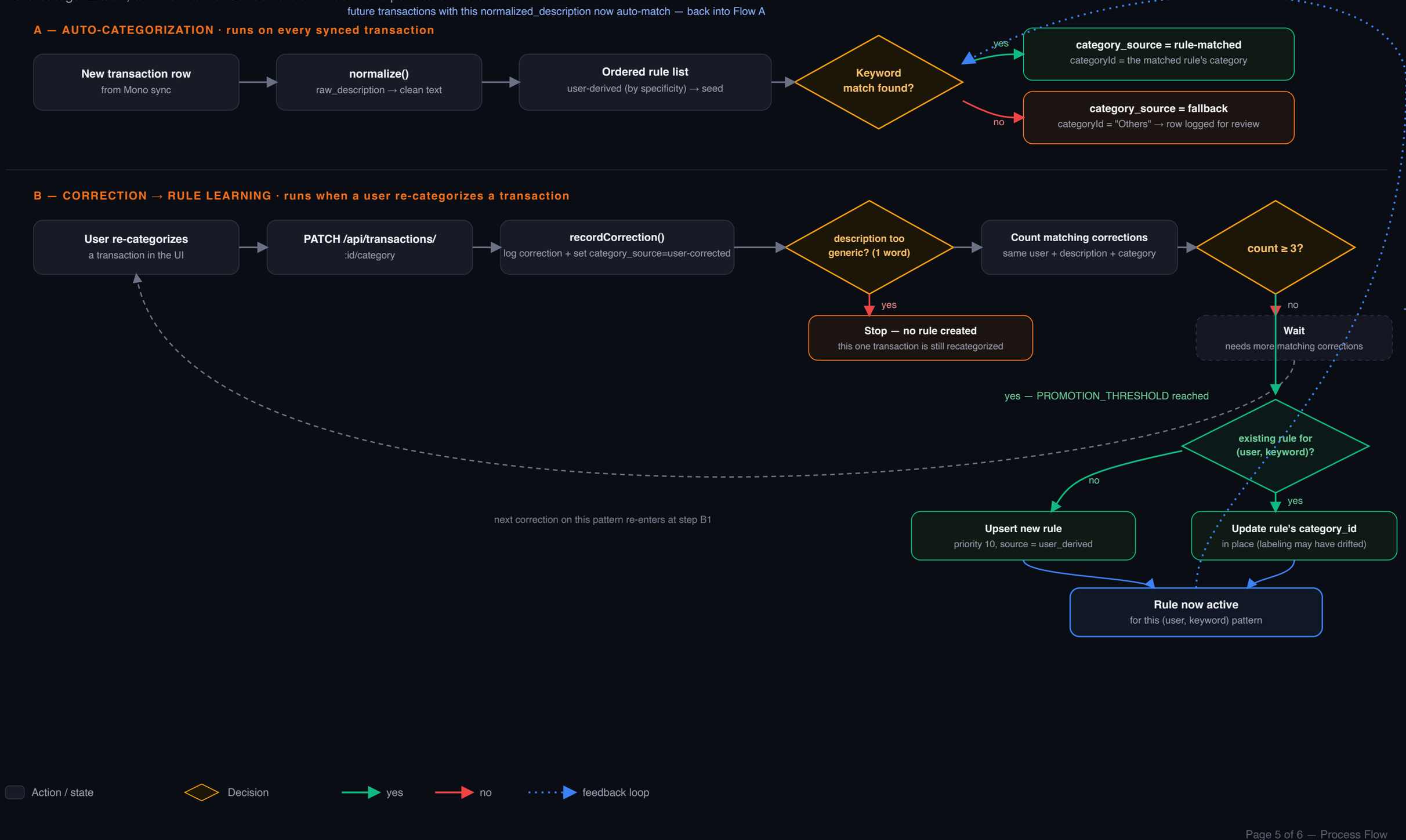
D6 notifications

refreshAll() closes the loop back to the UI

Process step · External entity · Data store · write / read to store · call to Mono API

Carrot — Process Flow

Auto-categorization, and how a manual correction becomes a permanent rule



Carrot — User Flow

From a cold landing to every core surface of the app